

Docker Swarm

Building services replicated on cluster

Introduction and
a working example

Virtualizzazione e Integrazione di Sistemi

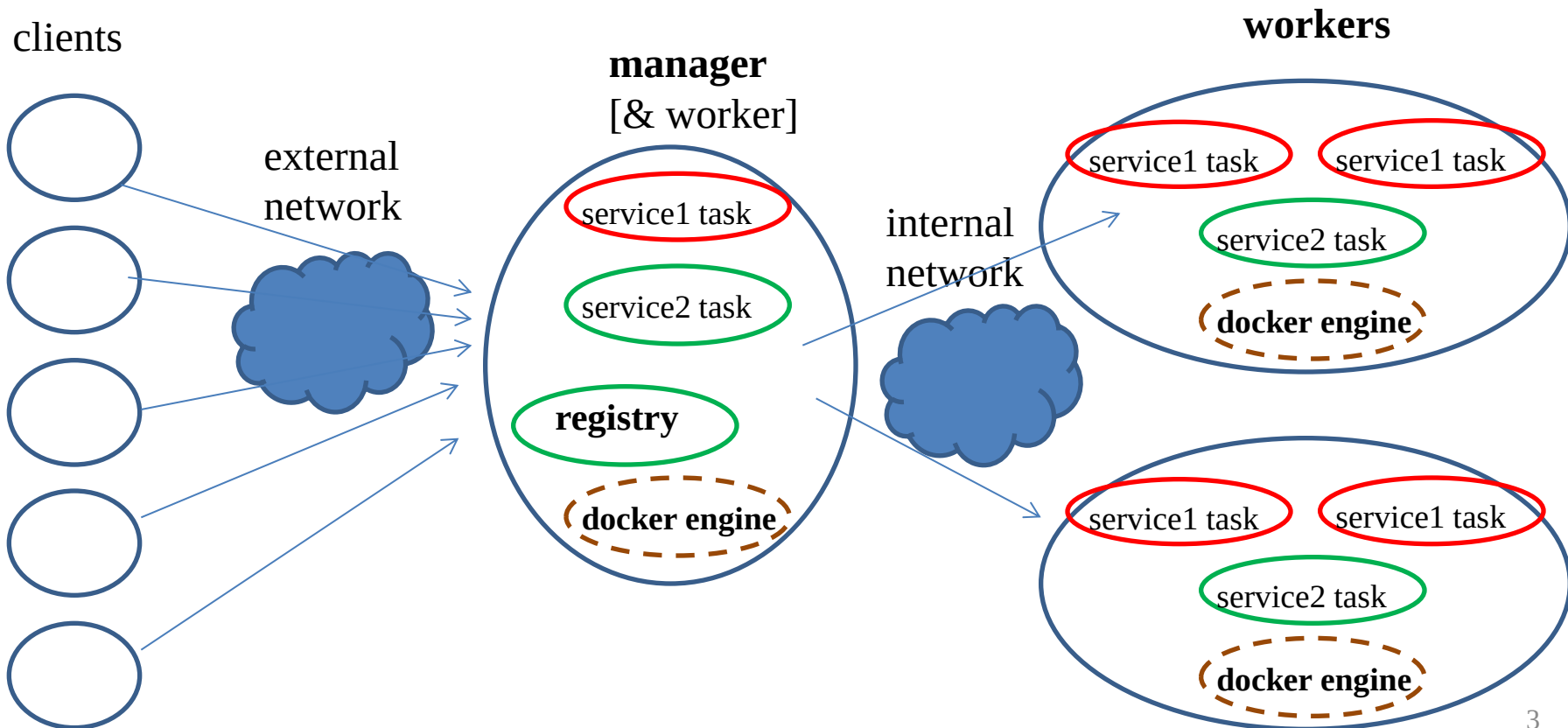
14/05/2025

The Swarm structure (1/2)

- A **swarm** is composed by one or more **nodes** (**machine**). A given node plays the role of docker swarm **manager**. Each other nodes play the role of **worker**. Thus, the manager node plays both the roles of manager and worker.
- The swarm need to be initialized by setting up the manager using the "docker swarm init" command in the manager node. This command changes the working "mode" of the docker engine of the node from "sigle node" to "swarm".
- A swarm can run a stack.
- A **stack** is a group of services.
- Each service is the service seen in Compose: a **service** is a container image and its structures (networks, volumes, ..).
- A service can be deployed over multiple workers of the swarm, creating multiple container instances (tasks) on each worker node. Each container instance is called **task**.
- Each task of a service is a **replica** of that service. A worker can hosts more replicas of a given service.
- Each worker can run more containers (replicas) of one or more services.

The Swarm structure (2/2)

- In essence, a **stack** is a set of **tasks** (working containers), grouped in **services**, running over a cluster of **worker** nodes and coordinated by a **manager** node.
- The manager coordinates the workers and distributes the task among the worker nodes. The manager may receive the client requests and distributes them to the service tasks instantiated on the workers.



Network assumptions

- Docker swarm manager uses iptables to forward the client connections to the selected replica.
- The swarm messages among manager and workers use the tcp ports 2376 and 2377.
- The swarm approach assumes the nodes are connected by a network.

Swarm operations (from the example)

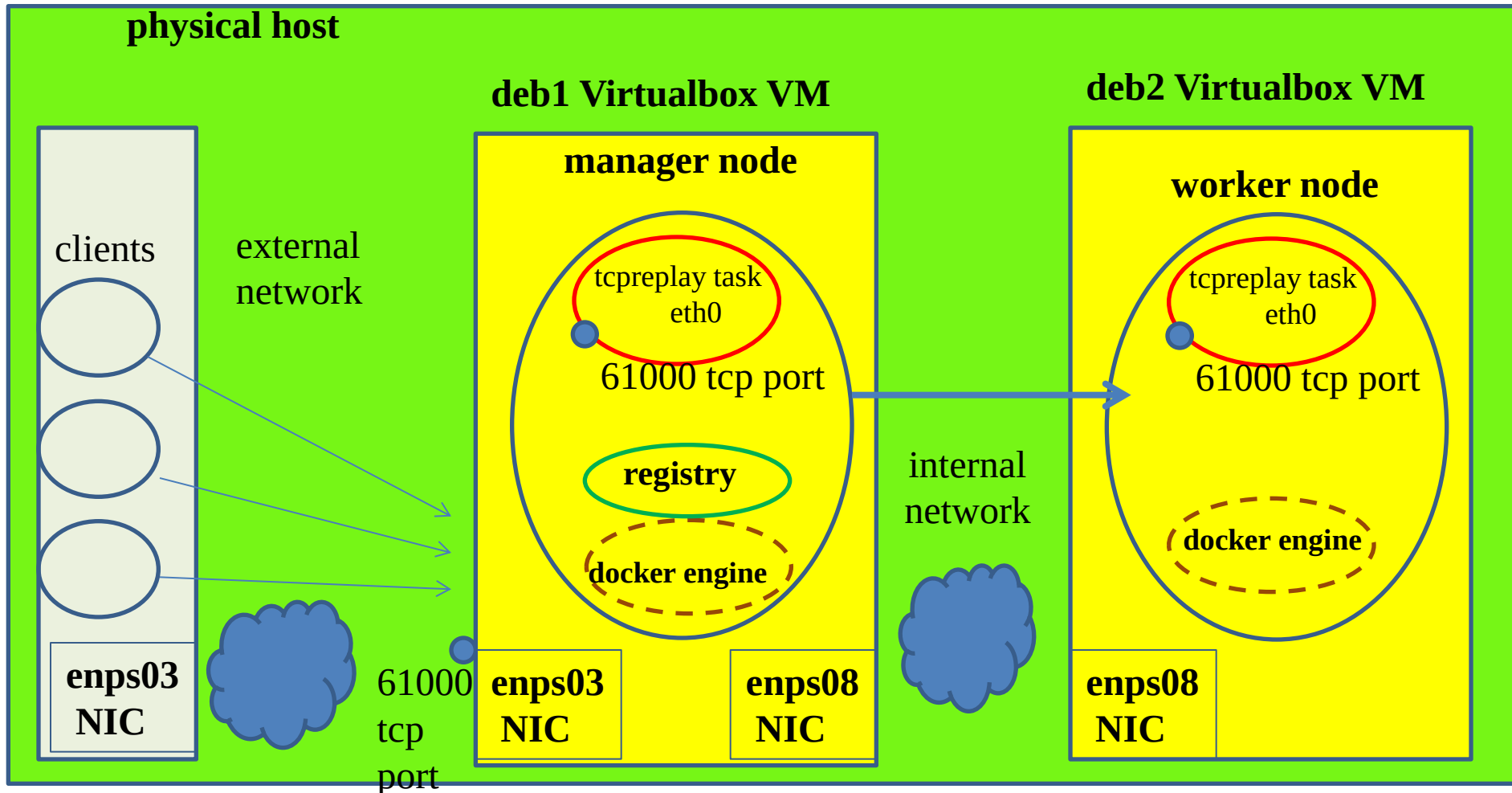
- **docker swarm init** --advertise-addr 10.133.7.101
- **docker service create** --name myregistry --publish published=5000,target=5000 registry:2
- **docker service ls**
- **docker stack deploy** --compose-file docker-compose-v3.yml mystack
- **docker stack services** mystack
- **docker swarm join** --token SWMTKN-1-4oj8bfuo7wk1jh0kgymq33xosmj5rv94zn3r89nfhaudr7290-73ilqpldlphdxopip9ibc3dpt 10.133.7.101:2377
- **docker service scale** mystack_tcpreplayservice=2
- **docker node ls**
- **docker node update** --availability drain node-1
- **docker swarm leave**
- **docker stack rm** mystack
- **docker service rm** mystack_tcpreplayservice
- **docker service rm** myregistry
- **docker swarm leave** --force

More information

- <https://docs.docker.com/engine/swarm/>

The example structure

- yellow: VM with swarm node
- beige: VM with client, emulates external clients.
- green: physical host



The files for the example

Client side

clitcp.c

MakefileClient

Server side

tcpreplay.c

Makefile

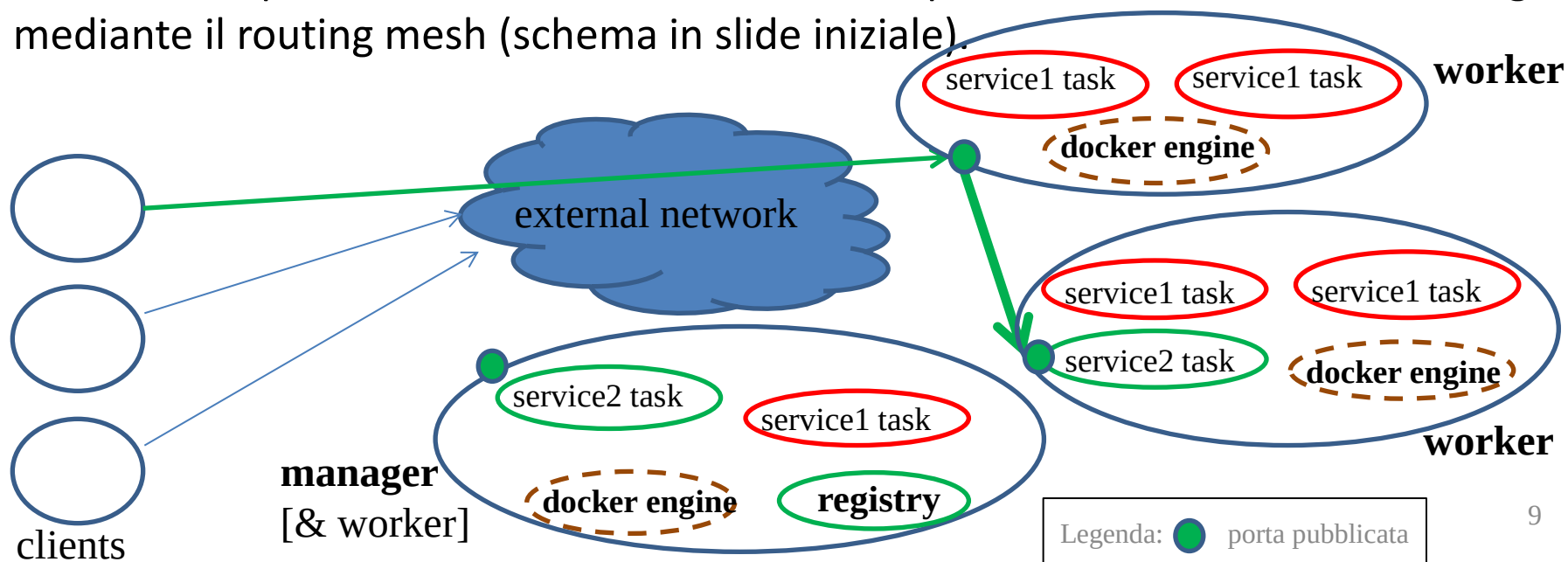
Dockerfile

docker-compose.yml

docker-compose-v3.yml

Pubblicazione porte in modo Ingress e Routing Mesh

- A rigore, non sarebbe strettamente indispensabile collocare i worker su una rete interna separata, poiché posso far sì che **tutti i nodi possano ricevere richieste per un servizio direttamente dall'esterno** chiedendo di usare un meccanismo di **pubblicazione delle porte** ● (ports: mode: ingress) che espone la porta di un servizio su **TUTTI i nodi** anche quelli che non implementano il servizio.
- La pubblicazione delle porte attiva un ulteriore meccanismo, detto **routing mesh**, che, **quando un nodo qualunque riceve una richiesta su una porta pubblicata per un servizio, inoltra la richiesta ad un nodo che implementa il servizio**.
- Ovviamente, è più sicuro se i nodi worker lavorano su una rete interna separata e fisicamente possono ricevere richieste solo per tramite del nodo manager mediante il routing mesh (schema in slide iniziale).



Problematiche di sicurezza

- Esporre all'esterno porte di tutti nodi è un **problema di sicurezza**, aumenta la superficie di possibili attacchi. Si potrebbe esporre il solo nodo manager.
- Tra l'altro, alcuni sistemi esterni che bilanciano il carico, degli **Ingress Controller appositi tipo Traefix** o più general purpose come NGINX, possono interfacciarsi con il nodo manager per sapere quali sono i nodi su cui girano i servizi richiesti. Si parla di fare **discovery automatica delle repliche attive** per sapere quali repliche **stanno effettivamente funzionando**. Per fare questo però, Traefix deve risiedere sullo stesso nodo manager per vedere il socket docker di tipo unix (in sostanza un file nel filesystem del manager) `/var/run/docker.sock` esposto dal nodo manager su cui fare interrogazioni. Si potrebbe esporre sul manager un socket TCP invece che Unix e tenere Traefix su un nodo esterno separato, ma esporre la porta TCP del manager è considerato quasi peggio.
- Anche **esporre il nodo manager è un problema di sicurezza** perché il nodo manager ha visione e conoscenza di tutto il cluster e, come detto, espone un socket (unix o tcp) con API REST per conoscere la situazione del cluster.
- Esporre **il solo nodo** manager è un problema di robustezza, se il nodo manager va in crash allora il servizio sul cluster diventa inutilizzabile. Si potrebbe replicare il manager ma a quel punto se un manager va in crash come facciamo a saperlo?
- *Cenni per approfondimento: usare Traefik o NGINX con Consul, etcd o Docker socket per fare discovery automatica delle repliche attive.*

Discovery automatica delle repliche attive

In un cluster Docker Swarm, puoi avere servizi replicati su più nodi. Ad esempio, un servizio webapp può avere 10 repliche distribuite tra nodi diversi.

- Un **reverse proxy** come **Traefik** o NGINX deve sapere:
 - quali container stanno girando,
 - dove (in quale nodo), - su quale porta,
 - se sono pronti per ricevere richieste.
- Questa è la service discovery: il proxy rileva dinamicamente i container e configura le rotte verso di essi senza che tu debba scrivere a mano le configurazioni ogni volta che cambiano.

Strumenti di discovery:

1. Docker Socket (/var/run/docker.sock)

Semplice, ma funziona solo su nodi manager perché hanno visibilità del cluster. Funziona anche con più nodi manager.

2. Consul o Etcd

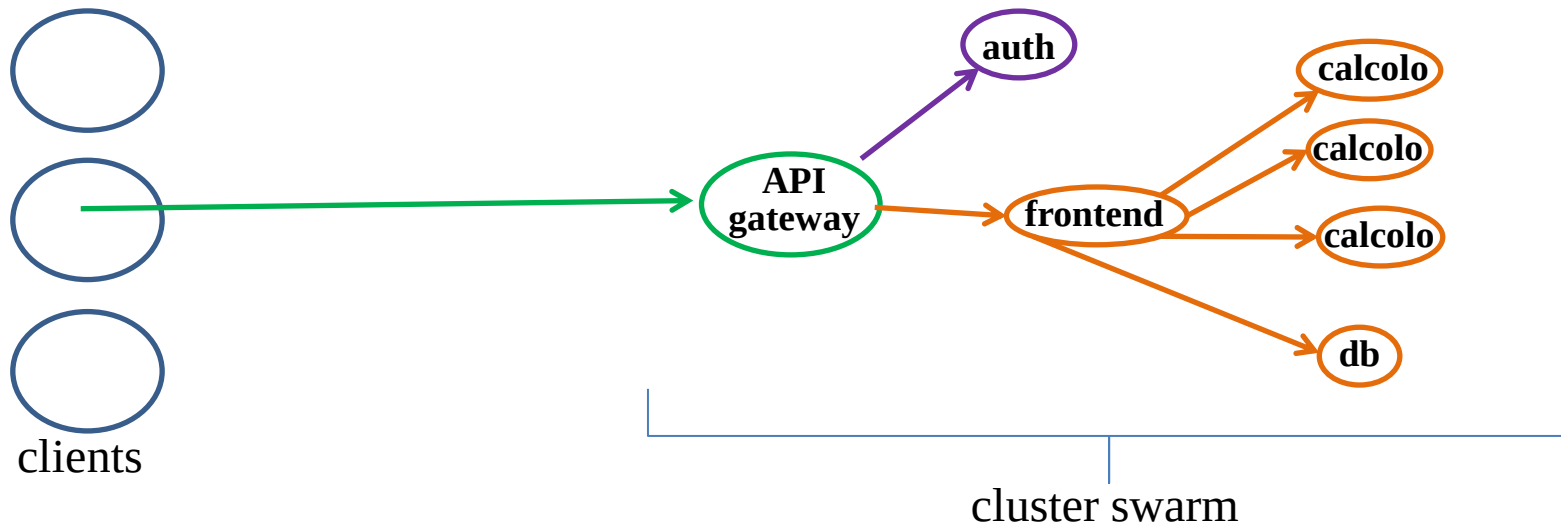
Sistemi esterni di key-value store usati per registrare servizi attivi. I container (o sidecar) notificano a Consul che sono vivi e su che indirizzo/porta stanno. Poi Traefik o NGINX leggono da Consul la mappa aggiornata. Questo approccio è più complesso, ma è decoupled da Docker, più portabile e può essere fault-tolerant.

Riepilogo: Discovery automatica delle repliche attive

Metodo	Accesso allo Stato del Cluster	Complessità	Auto-discovery	Note
Docker0 Socket	Solo da nodo manager	Bassa	yes	Facile, ma legato a Docker
Consul	Totale	Media/Alta	yes	Scalabile e multi-cluster
etcd	Totale	Alta	yes	Più usato in ambienti Kubernetes
Statico	Nessuno	Bassa	NO	Richiede configurazione manuale

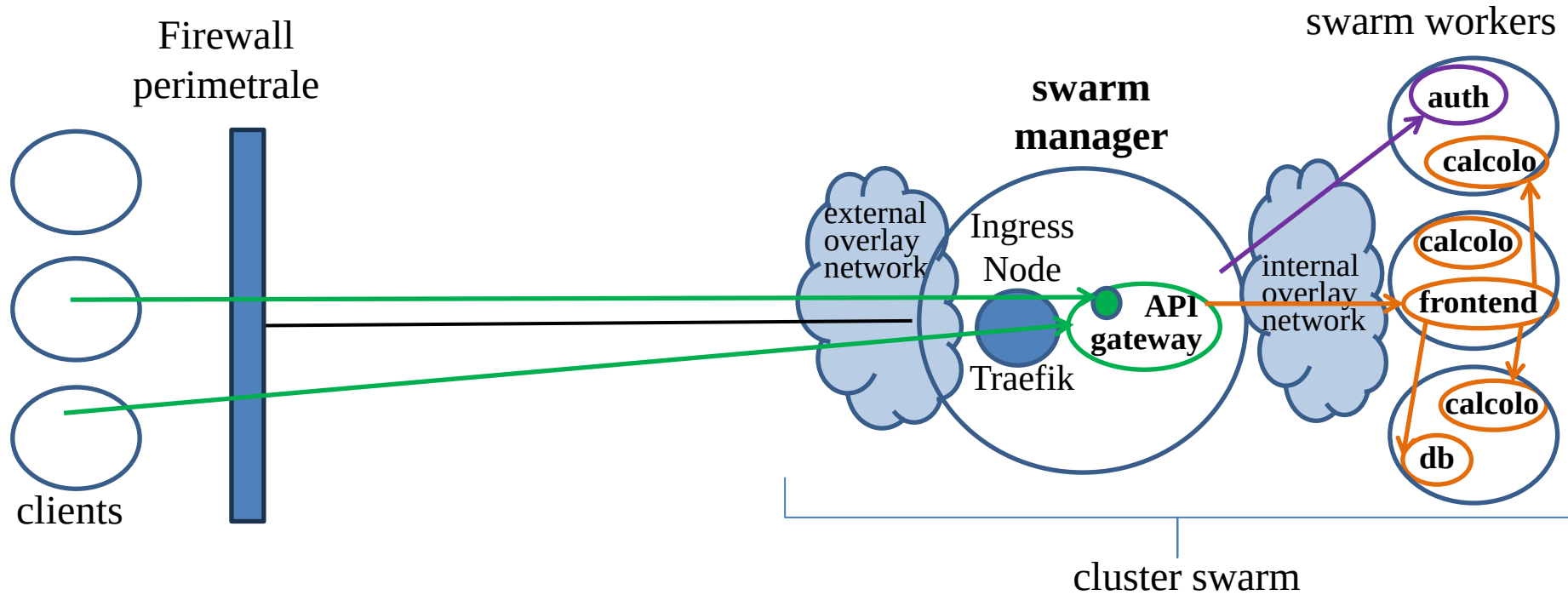
Dispiegare una applicazione su docker swarm

Struttura a microservizi



- Come strutturare il cluster swarm e come dispiegare l'applicazione?
- Criteri: sicurezza e robustezza

Architettura Semplice basata su docker swarm



- L'Ingress node Traefik risiede nel nodo manager per poter accedere al docker socket del nodo manager e poter così sapere dove stanno i container dei servizi richiesti e quali funzionano, per poi fare successive richieste ai singoli container nei nodi worker.
- Non c'è ridondanza, un solo manager swarm, ma posso replicare i container dei servizi perché Traefik li può rintracciare con il discovery dei servizi.
- Il docker socket di tipo unix è accessibile solo da un processo che stia nel nodo manager, perciò l'Ingress node traefik sta nel nodo manager.

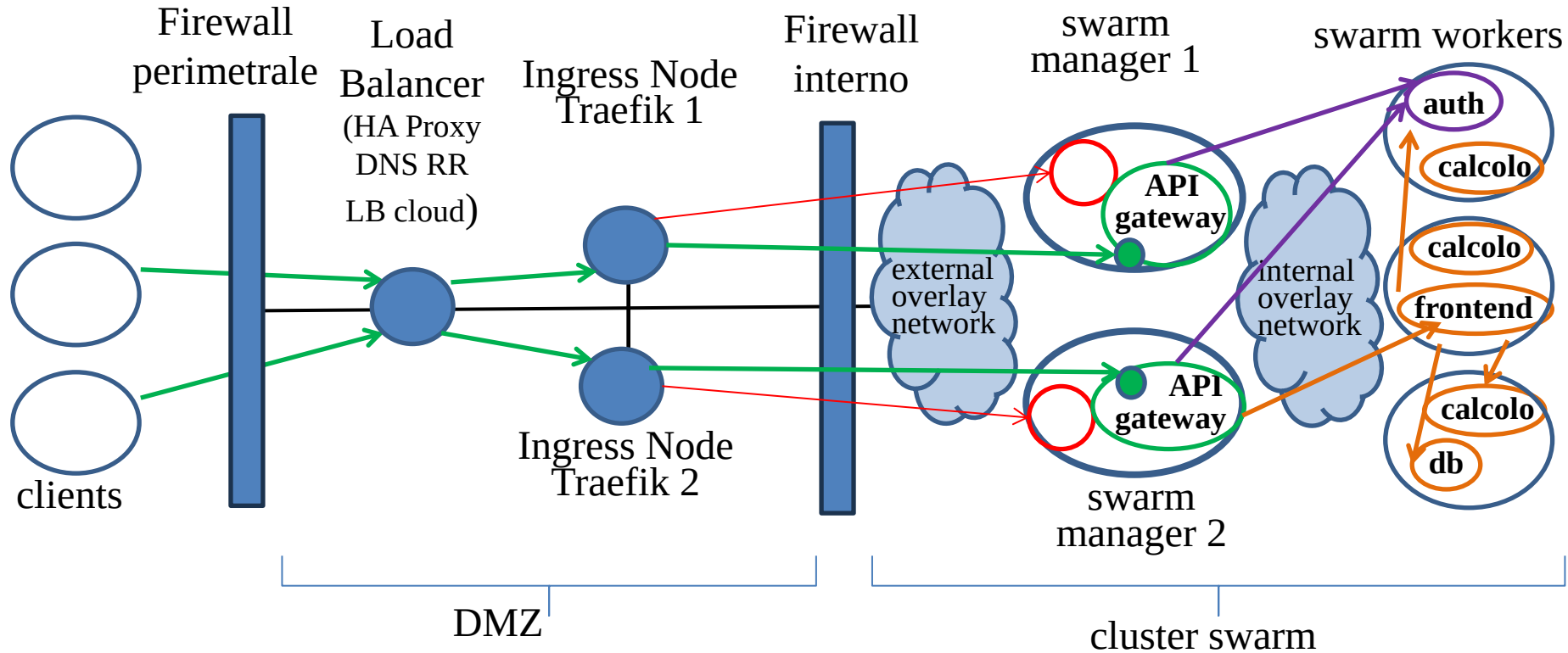
Ingress Node
Traefik



docker socket
unix



Architettura Sicura e Robusta basata su docker swarm



- La questione sicurezza risiede anche nel modo in cui viene realizzato la  comunicazione tra traefik e docker socket, con cui traefik chiede al nodo manager dove sono le repliche di container e quali funzionano, per poi fare successive richieste ai singoli container nei nodi worker.
- Nel manager, davanti al "docker socket" del manager posso mettere o **un socket proxy** configurato con https oppure **Consul**. Questi consentono accesso autenticato dall'esterno.
- Per affidabilità, ridondo mettendo due nodi manager, ciascuno con un suo Traefik.
- Devo mettere un Load Balancer che smista il traffico proveniente dall'esterno sui due Traefik.